Diseño e Implementación de una Unidad Aritmeticológica (ALU) de 4 bits en FPGA

Ingeniería en Sistemas, Comunicaciones y Electrónica, FES Cuautitlán – Gonzalez Jimenez Luis Ángel

Resumen

El presente documento detalla el diseño, simulación e implementación de una Unidad Aritmeticológica (ALU) de 4 bits basada en una arquitectura estrictamente combinacional. El sistema desarrollado es capaz de ejecutar ocho operaciones aritméticas y lógicas básicas (suma, resta, AND, OR, XOR, NOT, corrimiento a la izquierda y corrimiento a la derecha), además de calcular simultáneamente cuatro banderas de estado (acarreo/préstamo, desborde, cero y negativo) para caracterizar el resultado obtenido.

El desarrollo metodológico se llevó a cabo utilizando SystemVerilog como lenguaje de descripción de hardware dentro del entorno de Xilinx Vivado. El comportamiento lógico del circuito se validó inicialmente mediante un banco de pruebas (*testbench*), demostrando la correcta conmutación de las señales de salida ante diversos estímulos. Finalmente, el diseño fue sintetizado y descargado de manera exitosa en una tarjeta de desarrollo FPGA BASYS3, asignando los operandos a los interruptores físicos y los resultados a los indicadores LED. Las pruebas físicas y simuladas confirman que la unidad operó con precisión y sin latencias secuenciales no deseadas.

Introducción

Una Unidad Aritmeticológica (ALU) representa el núcleo computacional de cualquier procesador o sistema digital, siendo la encargada de procesar datos binarios mediante operaciones matemáticas y compuertas lógicas. En el ámbito del diseño de hardware, la implementación de una ALU permite comprender cómo las variables booleanas se transforman y rutean a través de circuitos puramente combinacionales para generar resultados exactos.

Para que un sistema mayor (como un microprocesador) pueda tomar decisiones basadas en estos cálculos, la ALU no solo debe entregar el valor resultante de la operación, sino también emitir señales de estado conocidas como banderas (*flags*). En este proyecto se evalúan cuatro condiciones fundamentales:

- Acarreo / Préstamo (c): Indica si una suma excede la capacidad de los bits disponibles o si una resta requiere un préstamo del bit continuo.
- Desborde (o): Evalúa si el resultado de una operación aritmética con signo (en complemento a dos) ha excedido el rango de representación positivo o negativo válido.
- Cero (z): Confirma si la totalidad de los bits del bus de salida se encuentran en un nivel lógico bajo.
- Negativo (n): Refleja el estado del bit más significativo, determinando el signo del resultado en aritmética de complemento a dos.

El desarrollo contemporáneo de estos sistemas depende de los Lenguajes de Descripción de Hardware (HDL). Utilizando el modelado a nivel de comportamiento (*behavioral*) en SystemVerilog dentro del entorno Vivado, es posible describir la lógica del circuito para su posterior síntesis física en una matriz de compuertas programables (FPGA), como la contenida en la tarjeta BASYS3. Este flujo de trabajo moderno facilita la simulación exhaustiva y asegura que las ecuaciones booleanas se traduzcan eficientemente en hardware real.

Diseño General

El diseño de la ALU se divide en una interfaz de nivel superior (*Top-Level*) y un bloque de lógica de enrutamiento interno, todo descrito mediante modelado de comportamiento en SystemVerilog para asegurar una síntesis puramente combinacional.

El sistema externo consta de dos buses de entrada para los operandos A y B (ambos de 4 bits), y un bus selector Operation de 3 bits. Las salidas incluyen un bus **Output** de 4 bits para el resultado y cuatro líneas independientes de 1 bit para las banderas de estado (c, o, z, n).

Internamente, la ALU centraliza sus operaciones en un bloque always_comb, el cual infiere un multiplexor de 8 vías a través de una estructura case. Dependiendo del código recibido en Operation, el sistema enruta los operandos hacia el circuito combinacional correspondiente (sumador, restador, compuertas lógicas simples o desplazadores).

Gestión de Datos y Acarreo

Para manejar las operaciones aritméticas sin perder información de desbordamiento de bits, se implementó un bus temporal interno de 5 bits denominado result_ext. Durante la suma y la resta, las operaciones algebraicas evalúan operandos de 4 bits y depositan el resultado en este bus. El quinto bit (el más significativo, result_ext[4]) captura automáticamente el acarreo (*carry*) o el préstamo (*borrow*). En las operaciones lógicas y de corrimiento, donde el acarreo no aplica, se realiza una concatenación forzando el quinto bit a 0 (por ejemplo, {1'b0, A & B}). Al final del bloque, los 4 bits menos significativos de result_ext se asignan a la salida Output.

Lógica de las Banderas de Estado (Flags)

Las banderas se evalúan en paralelo mediante lógica booleana y comparadores combinacionales en la etapa final del bloque:

- **Zero (z):** Un comparador de igualdad verifica si el resultado final coincide exactamente con 0000.
- **Negative (n):** En la aritmética de complemento a dos, el signo del número reside en el bit más significativo (MSB). Por lo tanto, la bandera **n** se conecta directamente al bit Output[3].
- **Carry (c):** Toma su valor directamente del bit extra calculado en las operaciones aritméticas, mapeándose a result_ext[4].
- **Overflow (o):** El desborde ocurre cuando el resultado de una operación aritmética con signo excede el rango representable con 4 bits. Se calcula evaluando los bits de signo (bit 3) de los operandos y del resultado. Para la suma, se implementó la siguiente ecuación booleana:

$$o = (\sim A[3] \cdot\sim B[3] \cdot \text{result_ext}[3]) + (A[3] \cdot B[3] \cdot\sim \text{result_ext}[3])$$

Para la resta, la lógica es idéntica, pero invirtiendo el bit de signo del operando B.

Implementación a nivel de compuertas (RTL)

El código SystemVerilog desarrollado es sintetizado por Vivado, traduciendo las expresiones lógicas de comportamiento en una red de compuertas lógicas, sumadores completos (*Full Adders*) interconectados y multiplexores. El diagrama RTL detallado que ilustra esta síntesis de componentes físicos se encuentra adjunto en los apéndices de este documento.

Análisis de Resultados

El análisis de las formas de onda generadas mediante la simulación en Vivado confirma que la Unidad Aritmeticológica opera correctamente en los dominios de tiempo y lógica combinacional. El banco de pruebas inyectó estímulos secuenciales evaluando las ocho instrucciones operativas y verificando las transiciones del bus de salida y las banderas de estado.

Durante las pruebas aritméticas, se validó el comportamiento del circuito bajo la representación binaria de complemento a dos. Al ejecutar la suma de los valores 1111 (15) y 0001 (1) bajo el código de operación 000, el resultado físico en el bus de 4 bits convergió en 0000, activando de manera simultánea y correcta las banderas de cero (z=1) y de acarreo (c=1). Para comprobar la respuesta ante una discrepancia de rango, se sumaron los operandos positivos 0111 (7) y 0001 (1). El sistema arrojó 1000 (-8 en decimal), encendiendo correctamente el indicador de desborde (o=1) y la bandera negativa (n=1) debido a la alteración del bit de signo. En las operaciones de resta (operación 001), el caso de 0010 (2) menos 0110 (6) generó el valor 1100 (-4 en complemento a dos), activando de manera esperada el préstamo en la bandera **c** y el indicador de resultado negativo.

Por su parte, las evaluaciones lógicas y de corrimiento demostraron un aislamiento adecuado de las banderas aritméticas. Al aplicar un corrimiento lógico a la derecha (operación 111) sobre el operando 0011, el bit menos significativo se desplazó hacia la salida, introduciendo un cero en el bit más significativo y produciendo el valor 0001. En este escenario, así como en las compuertas booleanas puras (AND, OR, XOR, NOT), las banderas **c** y **o** se mantuvieron forzadas en un nivel lógico bajo (0). Esto confirma que la lógica de rellenado de bits programada en SystemVerilog previno con éxito la activación errónea de los indicadores de acarreo o desborde en instrucciones estrictamente lógicas. Las transiciones entre operaciones ocurrieron sin retrasos perceptibles ni retención de estados previos, garantizando la ausencia de cerrojos lógicos (*latches*) en el diseño final.

Conclusión

El desarrollo de esta Unidad Aritmeticológica de 4 bits demostró la eficiencia y practicidad de utilizar SystemVerilog para el diseño moderno de circuitos digitales combinacionales. El uso de un modelado a nivel de comportamiento permitió estructurar el ruteo de las ocho operaciones y el cálculo preciso de las cuatro banderas de estado sin la necesidad de instanciar manualmente arreglos complejos de compuertas lógicas o sumadores completos.

La etapa de simulación mediante el banco de pruebas (*testbench*) resultó ser un paso crítico, ya que permitió observar las formas de onda y validar el comportamiento del acarreo, el desborde, el cero y el signo bajo las reglas de la aritmética de complemento a dos, reduciendo el riesgo de errores antes de proceder al hardware real.

Finalmente, la implementación física en la tarjeta BASYS3 confirmó el éxito integral del flujo de diseño. La respuesta inmediata de los LEDs al cambiar los operandos y la instrucción en los interruptores físicos comprobó que el uso riguroso de directivas como always_comb y la correcta inicialización de variables por defecto previnieron exitosamente la inferencia de *latches* o elementos secuenciales no deseados, garantizando un procesamiento de datos ágil, estable y totalmente combinacional.

Apéndices

Apéndice A. Código Fuente del Módulo ALU_4bit

```
`timescale 1ns / 1ps

module ALU_4bit(

    input  logic [3:0] A,
    input  logic [3:0] B,
    input  logic [2:0] Operation,
    output logic [3:0] Output,
    output logic n,
    output logic z,
    output logic o,
    output logic c
);

    logic [4:0] result_ext;

    always_comb begin

        c = 1'b0;
        o = 1'b0;
        result_ext = 5'b00000;

        case (Operation)
            3'b000: begin
                result_ext = A + B;
                c = result_ext[4];

                o = (~A[3] & ~B[3] & result_ext[3]) | (A[3] & B[3] & ~result_ext[3]);

            end

            3'b001: begin // A - B
                result_ext = A - B;
                c = result_ext[4];
                o = (~A[3] & B[3] & result_ext[3]) | (A[3] & ~B[3] & ~result_ext[3]);

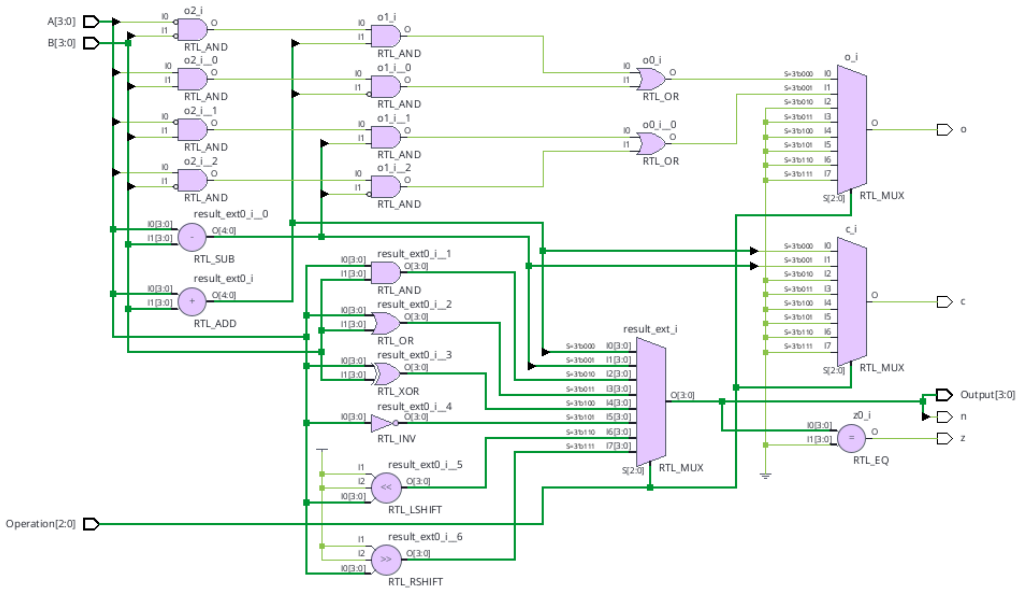
            end
            3'b010: result_ext = {1'b0, A & B};
            3'b011: result_ext = {1'b0, A | B};
```

```
            3'b100: result_ext = {1'b0, A ^ B};
            3'b101: result_ext = {1'b0, ~A};
            3'b110: result_ext = {1'b0, A << 1};
            3'b111: result_ext = {1'b0, A >> 1};
            default: result_ext = 5'b00000;
        endcase

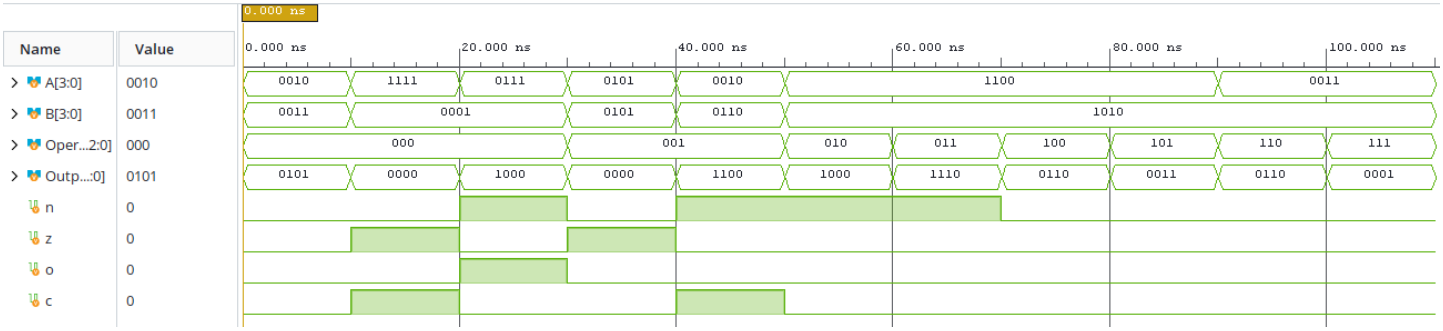
        Output = result_ext[3:0];
        n = Output[3];
        z = (Output == 4 [4:0] );
    end

endmodule
```

Apéndice B. Diagrama RTL



Apéndice C. Simulaciones en diagramas de tiempo



Bibliografía

[1] M. M. Mano y M. D. Ciletti, *Digital Design: With an Introduction to the Verilog HDL, VHDL, and SystemVerilog*, 6ta ed. Nueva York, NY, EE. UU.: Pearson, 2017.

[2] S. Sutherland, S. Davidmann y P. Flake, *SystemVerilog for Design: A Guide to Using SystemVerilog for Hardware Design and Modeling*, 2da ed. Nueva York, NY, EE. UU.: Springer, 2006.

[3] "Basys 3™ FPGA Board Reference Manual", Digilent, Pullman, WA, EE. UU., Doc. 502-182, 2016.

[4] "Vivado Design Suite User Guide: Logic Simulation", Xilinx, San Jose, CA, EE. UU., Doc. UG900, 2022.

[5] "Vivado Design Suite User Guide: Synthesis", Xilinx, San Jose, CA, EE. UU., Doc. UG901, 2022.